Лабораторная работа № 6

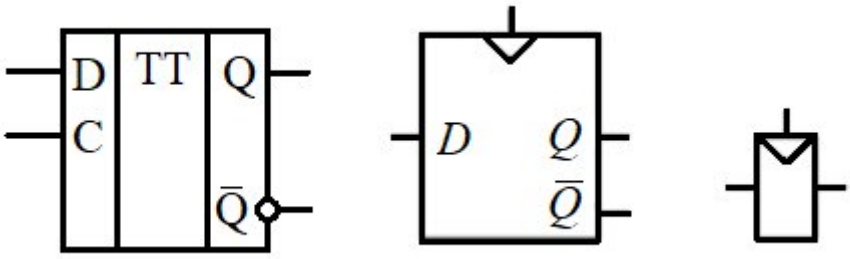
Триггеры и регистры

1. Триггеры

Триггер — простейшее последовательностное устройство, которое может находиться в одном из двух возможных состояний и переходить из одного состояния в другое под воздействием входных сигналов. Триггер является базовым элементом последовательностных логических устройств и служит для запоминания информации. Входы триггера разделяют на информационные и управляющие (вспомогательные).

D-триггером (от англ. delay) называют синхронный триггер, имеющий два входа: вход данных D и вход синхронизации C. D-триггер переключается по сигналу на C-входе в состояние, предписываемое D-входом.

Условное обозначение D-триггера показано на рисунке 1. В таблице 1 приведена таблица переходов триггера, в ней Q(t) – текущее состояние триггера, Q(t+1) – значение, записываемое в триггер после поступления сигнала на вход C.



а) б) в)

Рисунок 1 – Условно-графическое обозначение триггера а) российское б) европейское в) европейское сокращенное

Таблица 1. Таблица переходов D-триггера

C	D	Q(t)	Q(t+1)
0	x	0	0
0	x	1	1
1	0	x	0
1	1	x	1

D-триггеры бывают двух типов: D-защелка (latch) и D-триггер, синхронизируемый фронтом (flip-flop), обычно называемый D-триггером.

D-защелка применяется очень редко, поэтому в работе будет рассмотрен D-триггер, синхронизируемый фронтом. В нём состояние изменяется только в определенный момент времени. D-триггер синхронизируемый фронтом часто называют «D-триггер» или просто «триггер».

D-триггер копирует значение с D на Q по переднему фронту тактового импульса и помнит это состояние все остальное время. Часто передний фронт тактового импульса называют просто фронтом. Вход D определяет новое, будущее состояние триггера. Фронт определяет момент времени, когда состояние будет обновлено. Треугольник в обозначении указывает на то, что вход синхронизируется фронтом.

У многих триггеров выход $\overline{Q}$ отсутствует, и их обычно используют, когда $\overline{Q}$ не нужен.

У некоторых триггеров имеется еще один вход, называемый EN, или ENABLE (разрешить). Этот вход определяет, будут ли данные загружены по фронту или нет. Когда на EN подается логическая единица, то такой D-триггер ведет себя так же как и обычный D-триггер. Если же на EN поступает логический нуль, то триггер игнорирует тактовый сигнал и сохраняет свое состояние. Такие триггеры полезны, если мы хотим загружать значения в триггер только на протяжении какого-то времени, а не по каждому фронту тактового импульса. На рисунке 2 приведено обозначение такого триггера.

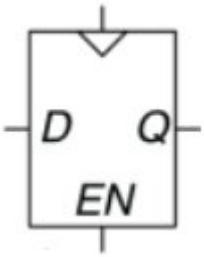


Рисунок 2 – Триггер с функцией разрешения, обозначение

В триггере с функцией сброса добавляется еще один вход, называемый RESET. Когда на RESET подан 0, сбрасываемый триггер ведет себя как обычный D-триггер. Когда на RESET подана 1, такой триггер игнорирует вход D и сбрасывает выход в 0. Триггеры с функцией сброса полезны, когда мы хотим ускорить установление определенного состояния (т.е. 0) во всех триггерах системы при первом включении.

Такие триггеры могут сбрасываться как синхронно, так и асинхронно. Синхронно сбрасываемые триггеры сбрасываются только по фронту сигнала CLK. Асинхронно сбрасываемые триггеры сбрасываются сразу же при поступлении логической единицы на вход RESET, вне зависимости от тактового сигнала.

Чаще в схемах применяются триггеры с синхронным сбросом. Условное обозначение такого триггера приведено на рисунке 3.

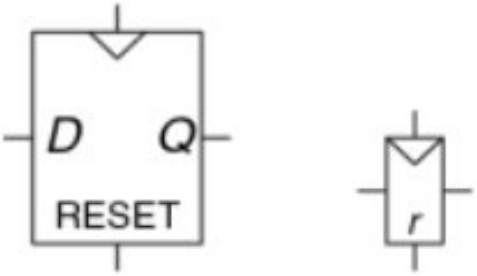


Рисунок 3 – Синхронно сбрасываемый триггер, обозначения

2. Регистры

N-разрядный регистр – набор из N триггеров с общим тактовым сигналом. Таким образом, все биты регистра обновляются одновременно. Регистр является ключевым блоком при построении большинства последовательностных схем. На рисунках 4,5 показана схема и обозначение 4-разрядного параллельного регистра со входами D3:0 и выходами Q3: 0. D3:0 и Q3:0 являются 4-разрядными шинами.

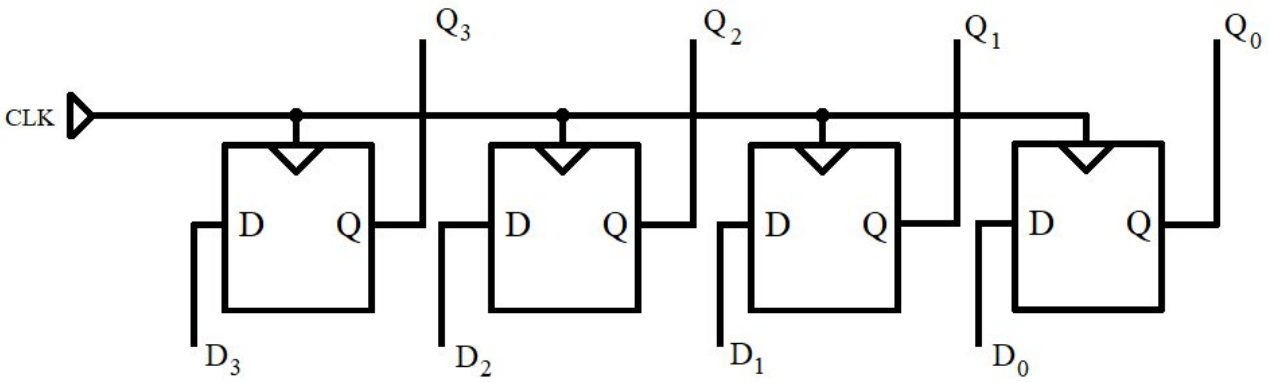


Рисунок 4 – Схема 4-разрядного регистра

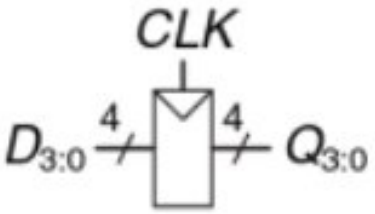


Рисунок 5 – Условное обозначение 4-разрядного регистра

На рисунке 6 показан сдвигающий регистр, который имеет вход тактового сигнала, последовательный вход S_{in}, последовательный выход S_{in} и N параллельных выходов $Q_{N-1:0}$. По каждому переднему фронту тактового импульса в первый триггер регистра записывается новый бит со входа Sin а содержимое следующих триггеров сдвигается вперед. Последний бит регистра можно считать с выхода Sout.

Сдвигающий регистр можно рассматривать как последовательно-параллельный преобразователь. На вход S_{in} поступают последовательные данные (по одному биту за раз). После N циклов последние N значений входного сигнала можно параллельно считать с выхода Q.

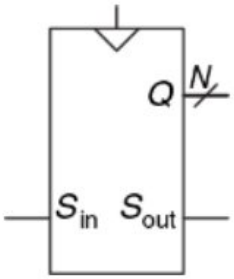


Рисунок 6 – Условное обозначение сдвигающего регистра

Как показано на рисунке 6, сдвигающий регистр может быть построен из N последовательно соединенных триггеров. Некоторые сдвигающие регистры имеют сигнал сброса для инициализации всех триггеров.

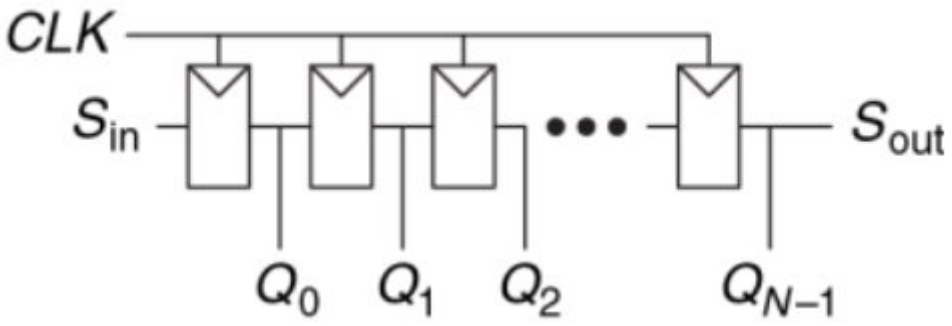


Рисунок 7 – Схема сдвигающего регистра

В параллельно-последовательный сдвигающий регистр параллельно загружается N бит, которые затем последовательно (по одному биты за раз) поступают на выход. Сдвигающий регистр можно модифицировать для выполнения как последовательно-параллельного, так и параллельно-последовательного преобразования, если к нему добавить параллельный вход $D_{N-1:0}$ и сигнал управления Load, как показано на рисунке 8. Когда вход Load активирован, во все триггеры параллельно загружаются данные со входа D. В противном случае сдвигающий регистр выполняет обычный сдвиг.

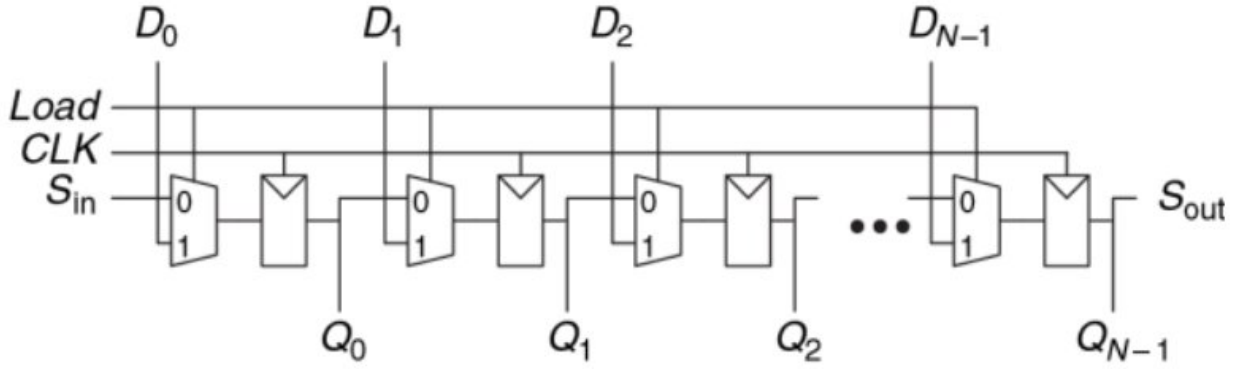


Рисунок 8 – Сдвигающий регистр с параллельной загрузкой

3. Распределитель импульсов

Используя триггеры, можно построить множество полезных схем. Одной из таких схем является распределитель импульсов. Распределителем импульсов называют устройство, с помощью которого временная последовательность импульсов, поступающая на его вход, распределяется по независимым выходным цепям.

Т.е. если последовательность импульсов поступает в моменты времени t_1, t_2, ..., t_k на один вход устройства, то в момент t_1 появляется импульс на выходе Q_1, в момент t_2– на выходе Q_2 и т.д. и в момент t_k появляется импульс на выходе Q_k.

Устройство работает циклически, т.е. импульс, появившийся на входе в момент t_{p+lk}, (где $p<k$, l-целое число) появляется на выходе Q_p.

На рисунке 9 приведена диаграмма следования импульсов на выходах распределителя. На рисунке 10 приведена схема распределителя импульсов на D-триггерах.

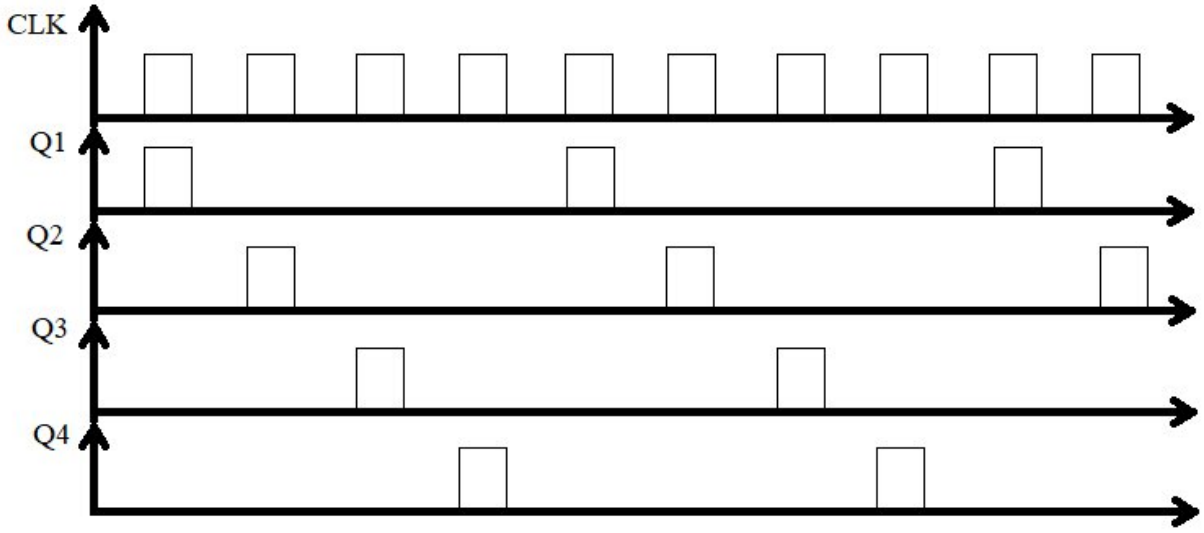


Рисунок 9 – Временная диаграмма следования импульсов

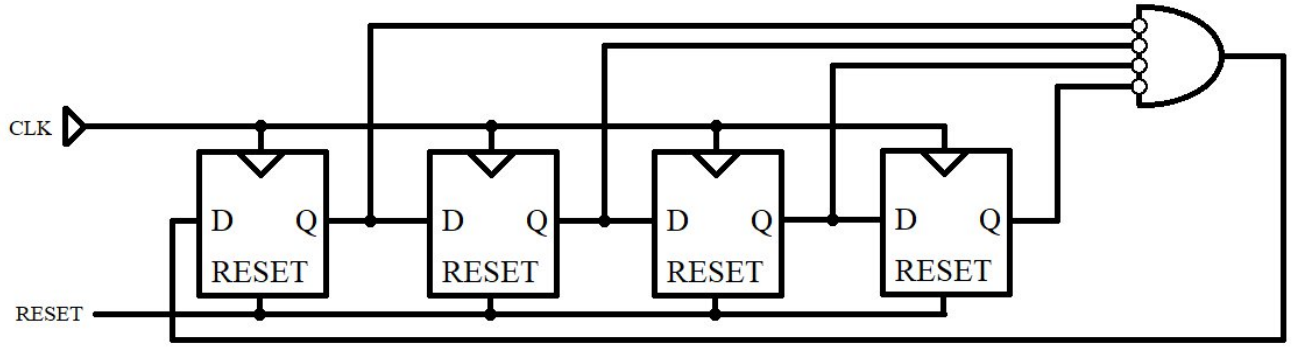


Рисунок 10 – Схема распределителя импульсов на триггерах

4. Описание на Verilog

Синтезаторы HDL распознают определенные идиомы и превращают их в конкретные последовательные схемы. Код, написанный в ином стиле, может правильно симулироваться, но в синтезированной схеме могут оказаться как грубые, так и труднораспознаваемые ошибки. В этом разделе представлены идиомы, рекомендованные для описания триггеров и регистров.

В листинге 1 приведена идиома для описания D-триггера, синхронизируемого передним фронтом.

Листинг 1 – Идиома для описания D-триггера

```
module flop(input logic clk,
                input logic d,
```

```
                output logic q);
        always_ff @(posedge clk)
            q <= d;
    endmodule
```

Как уже упоминалось ранее, в общем случае, оператор always языка SystemVerilog имеет вид:

```
always @(sensitivity list)
    statement;
```

Сигналы, значения которым присвоены в операторах always языка SystemVerilog, сохраняют своё состояние, пока не случится событие из списка чувствительности оператора, приводящее к изменению их значения. Поэтому код, использующий эти операторы с соответствующими списками чувствительности, может описывать последовательные схемы с памятью. Сигналы в списке чувствительности оператора always разделяются запятой или словом or.

Например, у триггера в списке чувствительности есть только сигнал clk, и потому триггер хранит старое значение q до следующего переднего фронта clk, даже если входной сигнал d изменился раньше.

В отличие от них, оператор непрерывного присваивания SystemVerilog (assign) перевычисляется каждый раз, когда изменяется какая-либо из переменных в правой части, поэтому этот оператор может описать только комбинационную логику.

Операторы always можно использовать для создания триггеров, защелок или комбинационной логики в зависимости от списка чувствительности и оператора. Из-за подобной гибкости языка при синтезе аппаратных блоков можно непреднамеренно получить нежелательную конфигурацию. Во избежание таких ошибок в SystemVerilog введены операторы always_ff, always_latch и always_comb. Оператор always_ff ведет себя так же, как always, но используется только тогда, когда подразумевается синтез триггеров, и позволяет инструментальной среде в противном случае выдавать предупреждение.

Операторы always могут содержать операторы if, за которыми может следовать оператор else. Если все возможные сочетания входов обработаны, то оператор синтезируется в комбинационную логику, иначе – в последовательную. В SystemVerilog операторы if обязаны быть внутри операторов always.

Триггеры со сбросом

В листинге 2 приведена идиома для описания триггера с синхронным сбросом.

Листинг 2 – Идиома для описания D-триггера с синхронным сбросом

```
module flopr(input logic clk,
                input logic reset,
        input logic  d,
                output logic q);
    // synchronous reset
```

```
     always_ff @(posedge clk)
          if (reset)
               q <= 1'b0;
          else
               q <= d;
endmodule
```

Триггеры с сигналом разрешения реагируют на тактовый импульс только при условии подачи активного уровня на линию разрешения. В листинге 3 показан триггер с условием en и синхронным сбросом reset, сохраняющий предыдущее значение, если оба сигнала имеют значение FALSE.

Листинг 3 – Триггер с условием и сбросом

```
      module flopenr(input logic clk,
               input logic reset,
               input logic en,
               input logic d,
               output logic q);
          // synchronous reset
          always_ff @(posedge clk)
               if (reset) q <= 1'b0;
               else if (en) q <= d;
      endmodule
```

Один оператор always можно использовать для описания нескольких элементов аппаратуры. Например, используя два последовательно подключенных триггера, можно построить простейший синхронизатор (рисунок 10).

Синхронизатор, является устройством, на вход которого поступает асинхронный сигнал D и тактовый сигнал CLK. За ограниченное время он формирует выходной сигнал Q, который с очень высокой вероятностью имеет корректный логический уровень.

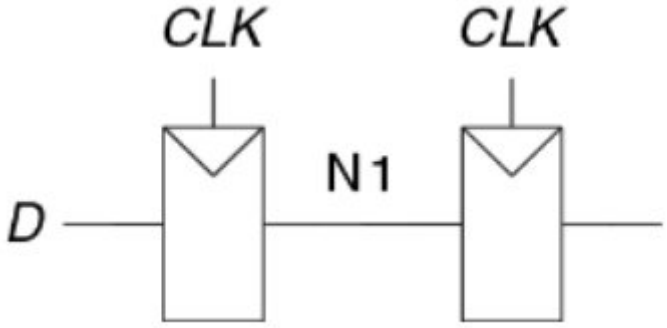


Рисунок 11 – Схема синхронизатора

В листинге 4 приведён код для описания простого синхронизатора. По переднему фронту clk, d копируется в n1, и в то же время n1 копируется в q.

Листинг 4 – Сихронизатор из двух триггеров

```
     module sync(input logic clk,
                    input logic d,
                    output logic q);
          logic n1;
```

```
        always_ff @(posedge clk)
        begin
            n1 <= d; // nonblocking
            q <= n1; // nonblocking
        end
    endmodule
```

В листинге 5 приведен код для описания четырёхразрядного параллельного регистра.

Листинг 5 – Параллельный регистр

```
    module flop(input logic clk,
                input logic [3:0] d,
                output logic [3:0] q);
        always_ff @(posedge clk)
            q <= d;
    endmodule
```

5. Задание

Задание 1:

1. Спроектируйте на языке SystemVerilog триггер, синхронизируемый по фронту.
2. Загрузите в плату, проверьте работоспособность. Объясните принцип работы.
3. Напишите тест, отладьте разработанную схему в симуляторе

Задание 2:

1. Спроектируйте на языке SystemVerilog триггер с синхронным сбросом. Объясните получившуюся схему.
2. Загрузите в плату, проверьте работоспособность. Объясните принцип работы.
3. Напишите тест, отладьте разработанную схему в симуляторе

Задание 3:

1. Спроектируйте на языке SystemVerilog триггер с разрешающим входом.
2. Загрузите в плату, проверьте работоспособность. Объясните принцип работы.
3. Напишите тест, отладьте разработанную схему в симуляторе

Задание 4:

1. Спроектируйте на языке SystemVerilog распределитель импульсов по схеме, приведённой на рисунке 10.
2. Напишите тест, отладьте разработанную схему в симуляторе.

Задание 5:

1. Спроектируйте на языке SystemVerilg четырёхразрядный сдвиговый регистр (по вариантам). Для этого модифицируйте код параллельного регистра. Для перераспределения битов можно использовать следующую конструкцию: `{data[2:0], 1'b0}`.

   1 вариант (нечётный номер бригады) – параллельно-последовательный

2 вариант (чётный номер бригады) – последовательно-параллельный

2. Напишите тест, отладьте разработанную схему в симуляторе